Matrix Chain Multiplication

Materia	Algoritmi
Anno	Secondo Anno
Data	@November 22, 2024
Studiato	Done

Ricordiamo che la moltiplicazione tra due matrici A, B è possibile se e solo se $A \in K^{p,q}$ e $B \in K^{q,w}$. La matrice risultante che si ottiene è $C \in K^{p,w}$. Ogni elemento di C è dato dal prodotto scalare vettoriale tra la riga i-esima di A e la colonna i-esima di B.

```
MATRIX-MULTIPLY(A,B, p, q, w):
    C = new-matrix(p * w)
    for i = 1 to p do
        for j = 1 to w do
            C[i,j] = 0
            for k = 1 to q do
                C[i,j] = C[i,j] + A[i,k] * B[k, j]
    return C
```

Tale procedura richiede $p \cdot w \cdot q$ operazioni per calcolare il prodotto tra le matrici.

La moltiplicazione tra matrici non gode della proprietà commutativa, ma di quella associativa. Supponiamo di avere tre matrici A_1, A_2, A_3 di dimensioni rispettivamente $10 \times 100, 100 \times 5, 5 \times 50$. Procedendo con $A_1 \cdot A_2$ e poi con $A_2 \cdot A_3$ si hanno in totale $5000 + 2500 = 7500$ prodotti da fare. $(A_1 \cdot A_2) \cdot A_3 = A_1 \cdot (A_2 \cdot A_3)$, ma se scelgo la seconda alternativa eseguo ben 75000 prodotti. Ossia, **la proprietà associativa ci dice che ci sono diverse strategie di scelta per come svolgere il prodotto**.
Risulta, quindi, necessario ridurre il numero di operazioni scalari. Quindi si tratta di **trovare il minor numero di operazioni**.

Matrici completamente parentesizzate

> *Una sequenza di matrici si dice **completamente parentesizzata** se è uguale ad una singola matrice o un prodotto tra due sequenze completamente parentesizzate tra parentesi.*

Quindi data S sequenza di matrici, essa è parentesizzata se è:

- una sola matrice A;
- un'espressione $(S_1 \cdot S_2)$ dove S_1, S_2 sono completamente parentesizzate.

Ad esempio:

$$(A_1 \cdot A_2) \cdot (A_3 \cdot A_4 \cdot A_5)$$

non è una sequenza completamente parentesizzata. Infatti $A_3 \cdot A_4 \cdot A_5$ non è completamente parentesizzata. Tuttavia:

$$(A_1 \cdot A_2) \cdot (A_3 \cdot (A_4 \cdot A_5))$$

Tale definizione è di **natura ricorsiva**. Inoltre, permette di modellare il problema della moltiplicazione tra matrici nel seguente modo: **trovare la migliore sequenza di matrici completamente parentesizzata**.

Nella valutazione di ogni sequenza, notiamo che **alcuni sottoproblemi vengono valutati più volte**. Infatti indicando con $P(n)$ il **numero di parentesizzazioni possibili di una sequenza di n matrici**:

- quando $n = 1$, la sequenza è costituita da una sola matrice;
- quando $n \geq 2$, abbiamo almeno due matrici, e un prodotto di matrici completamente parentesizzato è il prodotto di due sottoprodotti completamente parentesizzati di matrici, scegliendo una matrice k che divide la sequenza completa di matrici in due sequenze.

Dalle considerazioni precedenti si ottiene la ricorrenza:

$$P(n) = \begin{cases} 1 & n = 1 \\ \sum_{k=1}^{n-1} P(k)P(n-k) & n \geq 2 \end{cases}$$

e il numero di soluzioni è un $\Omega(2^n)$.

Risolvere il problema ricorsivamente

Sia data la sequenza $A_i...A_j$. La moltiplicazione della sequenza si può ottenere come il prodotto tra le sequenze $(A_i...A_k) \cdot (A_{k+1}...A_j)$ per qualche k, utilizzato come **punto per dividere la sequenza**. Continuando, si costruisce un albero binario le cui foglie sono sequenze $A_i...A_j$ tali che $i = j$. Ad ogni passo si sceglie un valore k e si suddivide in base al valore k scelto. L'obiettivo è scegliere un k ottimale.

Verifichiamo se tale problema gode del problema di sottostruttura ottima. Significa dire che se il numero di moltiplicazioni per calcolare la matrice $A_i \cdot ... \cdot A_j$ è $m(i,j) = m(i,k) + m(k+1,j) + \#moltiplicazioni$ è soluzione ottima, allora anche $m(i,k)$ è ottima e $m(k+i,j)$ pure. Per assurdo se supponiamo che $m(i,k)$ non sia ottima, avremo che $m'(i,k) + m(k+1,j) < m(i,j)$ ma ciò è un assurdo perché va contro la ipotesi che $m(i,j)$ è ottima.

Creiamo una funzione di costo. Se la nostra sequenza è costituita da A_i e basta, allora $k = i$. Generalmente, data la sequenza $A_i...A_j$, possiamo scegliere un valore di k per dividere tale sequenza nelle sequenze $(A_i...A_k) \cdot (A_{k+1}...A_j)$ tale che:

$$i \leq k < j$$

Per tenere conto delle dimensioni delle matrici $A_1...A_n$, abbiamo bisogno di $n+1$ valori. In particolare, P_0 è il numero di righe di A_1, P_1 è il numero di colonne di A_1 e A_2 ha un numero di righe pari a P_1, ossia un numero di righe pari al numero di colonne di A_1. Quindi generalmente la matrice di dimensione A_i è di dimensioni $P_{i-1} \times P_i$. Inoltre, il calcolo del prodotto tra le matrici $m[i,k]$ e $m[k+1,j]$ richiede $P_{i-1} \cdot P_k \cdot P_j$ prodotti scalari.
La soluzione del problema ricorsivo è data da:

$$m[i,j] = \begin{cases} 0 & i = j \\ \min\limits_{i \leq k < j} (m[i,k] + m[k+1,j] + P_{i-1} \cdot P_k \cdot P_j) & i < j \end{cases}$$

Il numero di sottoproblemi inoltre è dato dal numero di coppie di indici i, j tali che $1 \leq i \leq j \leq n$. Questo numero di coppie (non ordinate) è $\binom{n}{2} + n = \frac{n(n-1)}{2} + n = \Theta(n^2)$. Questo perché dobbiamo capire la parentesizzazione migliore per le sequenze del tipo $A_i ... A_j$.

Per risolvere il problema in forma dinamica utilizziamo una matrice invece di un array perché è necessario avere due indici i, j.

Esempio

Supponiamo di avere A_1, A_2, A_3 di dimensioni $10 \times 100, 100 \times 5, 5 \times 50$. Allora abbiamo una matrice 3×3. Allora il vettore P delle dimensioni delle matrici è:

$$P = [10, 100, 5, 50]$$

Costruiamo la matrice 3×3:

m	1	2	3
1	0	5000	7500
2		0	25000
3			0

Costruiamo le soluzioni per i sottoproblemi di dimensione 2. Questi sono i sottoproblemi $m[1,2]$ e $m[2,3]$. Allora si ha:

$$m[1,2] = 5000$$
$$m[2,3] = 25000$$

Infatti $m[1,2] = m[1,1] + m[2,2] + 10 \cdot 100 \cdot 5 = 0 + 0 + 5000$. Analogamente per $m[2,3]$ si ottiene $m[2,3] = m[2,2] + m[3,3] + 100 \cdot 5 \cdot 50 = 0 + 0 + 25000$.

Il problema di dimensione 3 è uno solo: $m[1,3]$, che con $k = 1$ e $k = 2$, ossia con tutte le suddivisioni possibili, ha soluzione 7500.

Le parti in cui $i > j$ non vengono considerate in quanto abbiamo imposto che $i \leq k < j$.

Procedura per la risoluzione del problema

La procedura Matrix-Chain-Order restituisce il numero minimo di operazioni da svolgere:

```
MCO(p,n):
   m = new-matrix(n * n)
   for i = 1 to n do
      m[i,i] = 0 // risolviamo i problemi di dimensione 1
   for l = 2 to n do // l = lunghezza della catena di matrici
   // risolviamo il problema di lunghezza l per risolvere quelli più grandi
      for i = 1 to n-l+1 do // la catena inizia da A_i
         j = i + l - 1      // e termina con A_j
            m[i,j] = +inf
            for k = i to j - 1 do
               if m[i,k] + m[k+1,j] + p[i-1] * p[k] * p[j] < m[i,j] then
                  m[i,j] = m[i,k] + m[k+1,j] + p[i-1] * p[k] * p[j]
   return m[1, n] // soluzione al problema A1...An
```

La complessità di tale procedura è $O(n^3)$.

Alla fine della procedura si conosce il numero minimo di operazioni da svolgere. Ma come si fa a **ottenere la parentesizzazione per tale costo minimo**?

Una volta aver ottenuto il valore k, devo mantenere una matrice S di dimensione $n \times n$ dove nella posizione $S[i, j]$ si memorizza il valore k tale che $m[i, j] = m[i, k] + m[k + 1, j] + P_{i-1}P_kP_j$.

Possiamo così ricostruire la struttura della parentesizzazione è la seguente:

```
PCO(i, j, S):
   if i=j then
      print Ai
   else
      k = S[i,j]
```

```
        print "("
        PCO(i, k, S)
        PCO(k+1, j, S)
        print ")"
```

Questa procedura stampa la struttura della parentesizzazione. Ad esempio, con le tre matrici dell'esempio precedente, si ha prima $PCO(1,3)$, e si stampa la parentesi aperta. Poi una chiamata ricorsiva $PCO(1,2)$ e $PCO(3,3)$. $PCO(1,2)$ stampa un'altra parentesi aperta e chiama $PCO(1,1)$ e $PCO(2,2)$. $PCO(1,1), PCO(2,2)$ sono dei casi base. Quindi stampano A_1, A_2. Viene stampata una parentesi chiusa, e poi viene eseguita la chiamata $PCO(3,3)$. $PCO(3,3)$ stampa A_3. Si ottiene così $((A_1A_2)A_3)$.

Volendo invece di utilizzare una matrice S possiamo utilizzare la matrice di risoluzione del problema m e salvare i valori k utilizzando i posti $m[j,i]$:

```
PCO(i, j, m):
    if i=j then
        print Ai
    else
        k = m[j,i]
        print "("
        PCO(i, k, S)
        PCO(k+1, j, S)
        print ")"
```